

mini color nn

A Complete Color Quantization of CIFAR-10 Dataset for CNN Image Classification

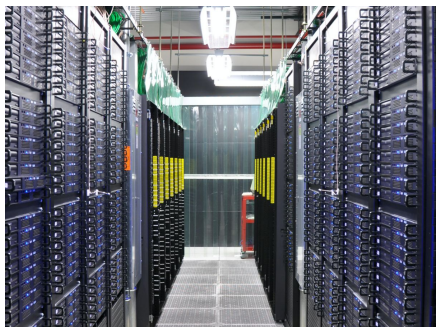
Sophia Huang

May 21, 2025 | tjSTAR

Computer Systems Lab | Yilmaz

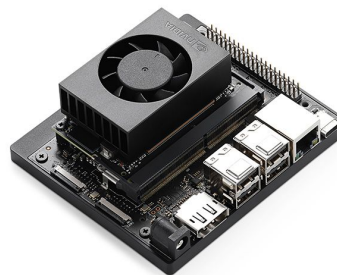
Problem

- Neural Networks are rapidly advancing and being widely applied across fields
- CNNs require a large number of parameters and are computationally intensive
- NNs rely heavily on high-performance hardware, limiting their usability on lower-resource devices like mobile and IoT (K. Ando et al., 2018)
- Restricts the extension of CNN applications



Data centers with racks of GPUs

VS

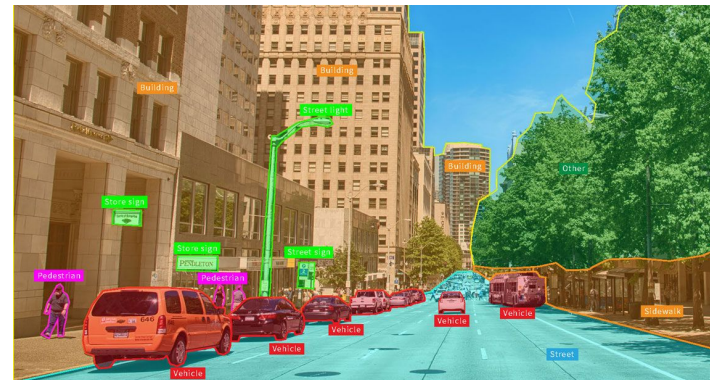


NVIDIA Jetson for IoT

Problem

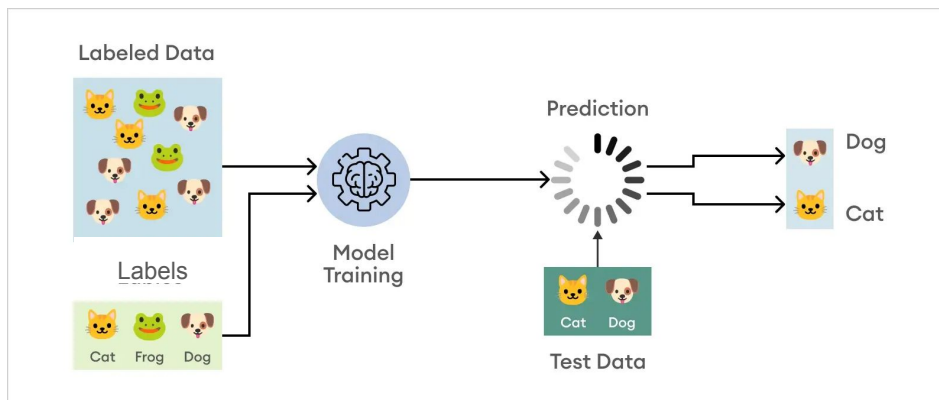
- Image classification involves high-dimensional data that requires significant computational resources (K. Ando et al., 2018)
 - E.g. matrix operations, multiple convolutional layers
- This reliance makes it challenging to run CNNs on low-resource devices with limited memory and processing power
 - Thermal Cameras
 - Surveillance Cameras

Need to find a method to alleviate the computational demand of image classification with neural networks.



Background: Image Classification with AI

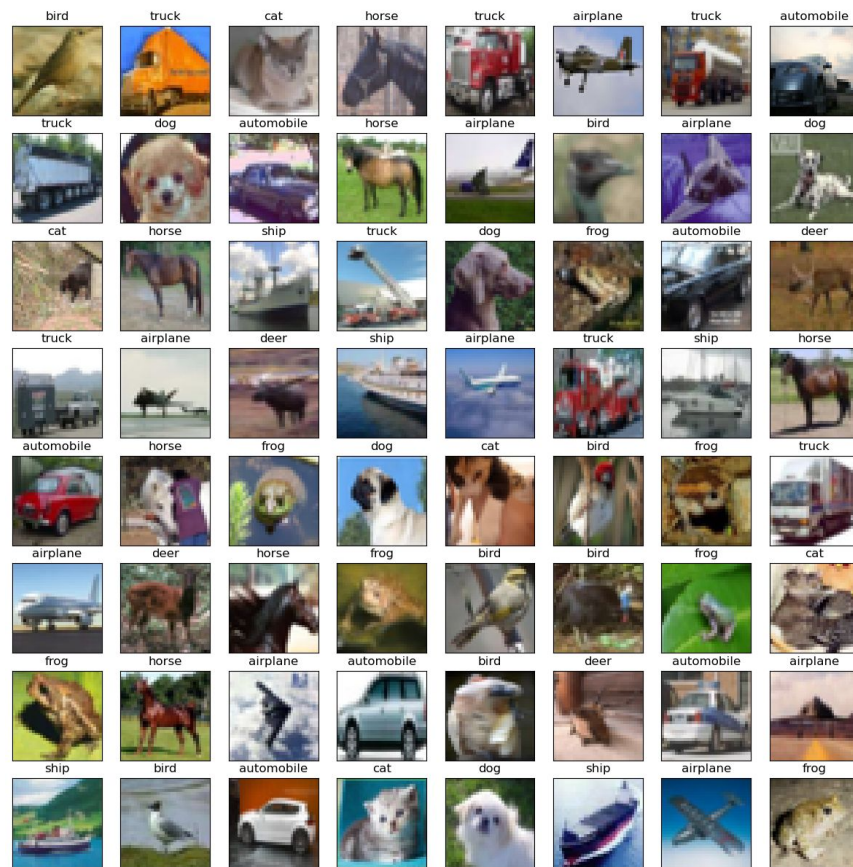
- Assigning a label or tag to an entire image based on preexisting training data of already labeled images
- **Algorithms + Image Datasets** are the essential components
- Pixel-level image analysis to determine the most appropriate label for the overall image



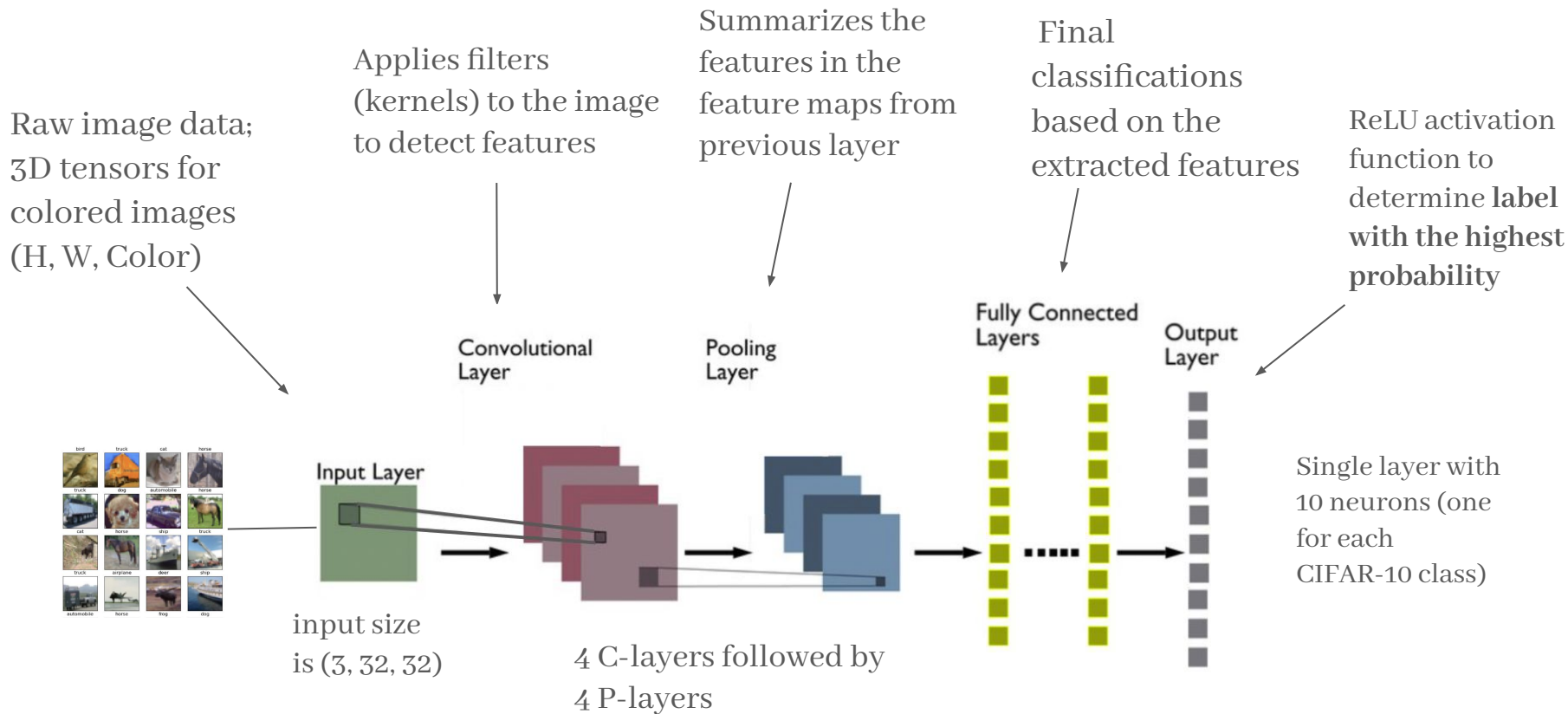
Source: <https://www.superannotate.com/blog/image-classification-basics>

Background: Dataset- CIFAR-10

- 60,000 color images, each of size 32x32 pixels, the dataset is segmented into 10 distinct classes (labels)
- Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck
- 6,000 images per class
- 50,000 are designated for training
- Remaining 10,000 are set aside for testing



Background: Algorithm- Convolutional Neural Networks (CNNs)



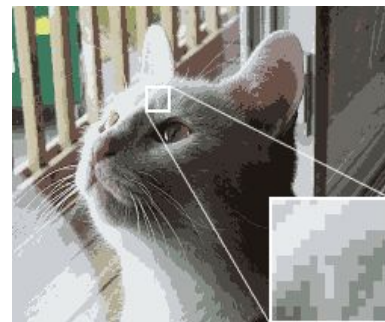
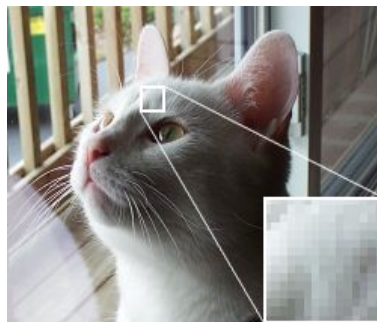
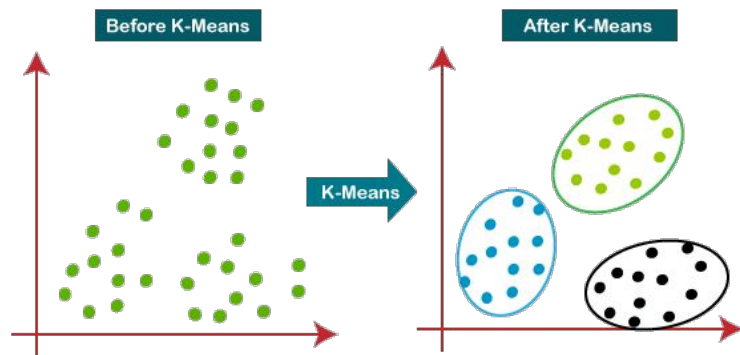
Images

Millions of unique possible RGB color values! Thousands across the whole CIFAR-10 dataset of 32×32 pixels.



Background: Vector Quantization using K-Means

- A partitioning algorithm that divides a dataset into k distinct clusters based on feature similarity.
 - Randomly select k initial centers
 - Assign each data point to the nearest centroid, forming clusters.
 - Recalculate centers by taking the mean of all points in each cluster.
 - Repeat until convergence
- Treat each pixel's RGB as a data point in a multidimensional space.
- Apply k -means clustering to group similar colors



Background: Dithering Using Floyd-Steinberg

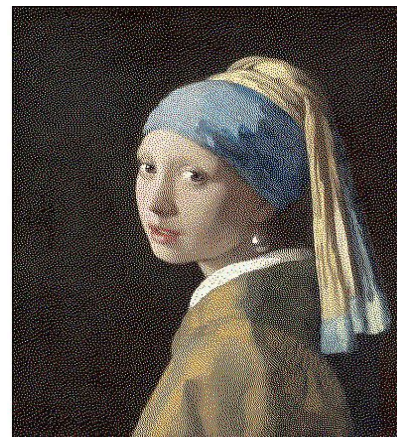
- The difference between the original and selected color (quantization error) is calculated and diffused
- Diffusion matrix

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{array}$$

- Only using colors in the limited palette, but dotted throughout to create the illusion
- Image is processed one pixel at a time, scanning from top-left to bottom-right.

Source: <https://scipython.com/blog/floyd-steinberg-dithering/>

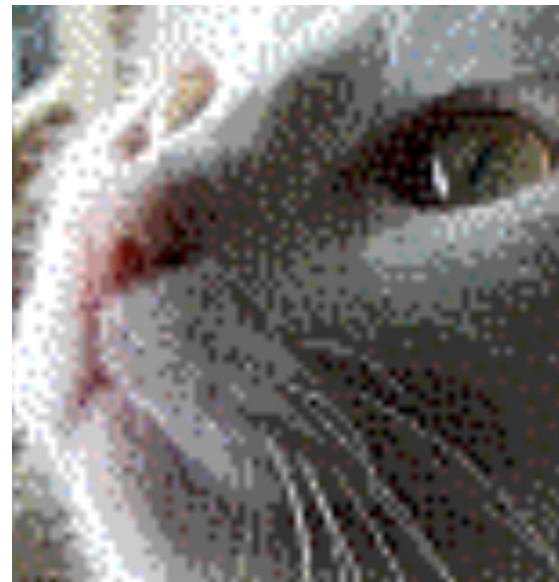
Example k=6



Background: Hypothesis

- Reducing the color palette might make it harder for the CNN to detect features
- But CIFAR-10 images are smaller
- Reduction in pixel complexity, so potentially simpler data processing and fewer computations in early layers
- Can result in smaller feature maps and reduced memory usage

Color Quantization and Dithering CIFAR-10 will make the CNN more efficient.



Other Solutions

- Dither NN: An Accurate Neural Network with Dithering for Low Bit-Precision Hardware (K. Ando et al., 2018)
 - MNIST and ImageNet-12 image datasets
 - Over 15× compression and 2× acceleration ratio
 - With no more than 1% loss in the classification accuracy
 - The “w/ Dither” model achieved 89.14% accuracy, whereas the baseline “w/o Dither” model was 85.83% accurate
- Quantized CNN: A Unified Approach to Accelerate and Compress Convolutional Networks (2017)
 - 1.5× acceleration and 2 to 3× compression
 - Accurate image classification can even be directly carried out on mobile devices within 1s

Novelty

- Uses CIFAR-10 dataset
- Uses Floyd-Steinberg Dithering algorithm
 - First project that uses error-diffusion technique
- Makes complete color quantized dataset publicly available

Why is mine better?

- Propagates quantization error to neighboring pixels, so higher quality diffusion
- Public Licensing on dataset so other researchers can use

Impact & Applications

- Impact

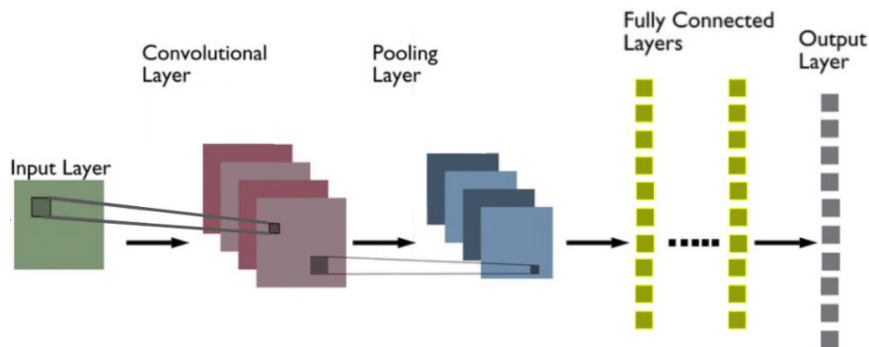
- Lower storage requirements for color-reduced images, especially useful for edge devices with limited memory
- Can speed up neural networks if the input is more lightweight
- Output dataset can be used in other research projects

- Applications

- Optimizes real-time image processing in low-power sensor cameras for firefighters
- Efficiency of surveillance cameras with faster image classification and reduced data storage

Method: System Architecture

- Load CIFAR-10 dataset (32x32 images in 10 classes).
- Apply VQ + FSD techniques to reduce color palettes (e.g., 16, 32, or 64 colors).
- Save quantized images for training and testing
- Pass new image dataset into CNN



 PyTorch

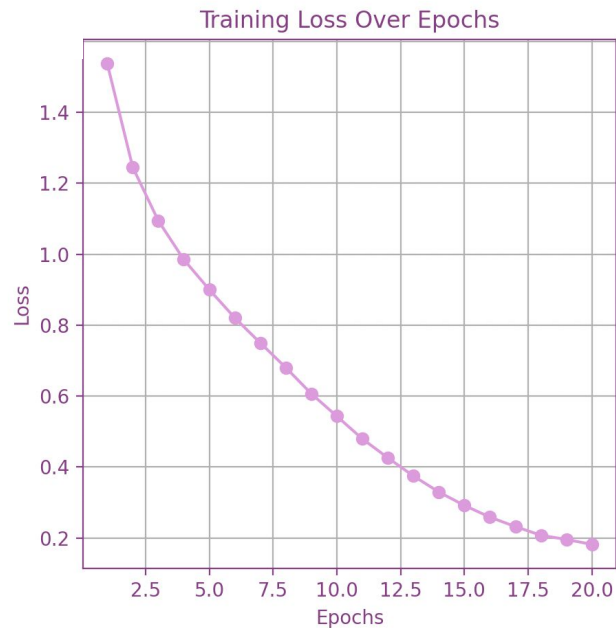
Results

Size, k of Image	Original Image	Modified Image	Quantize Time (s)	Dither Time (s)
32 x 32, k=8			0.072	0.008
32 x 32, k=8			0.201	0.014

Results

Size, k of Image	Original Image	Modified Image	Quantize Time (s)	Dither Time (s)
32 x 32, k=27			16.999	0.278
32 x 32, k=27			0.978	0.270

Baseline Results- CNN Data with CIFAR-10



Progress over first 20 epochs

CNN with **original CIFAR-10**:

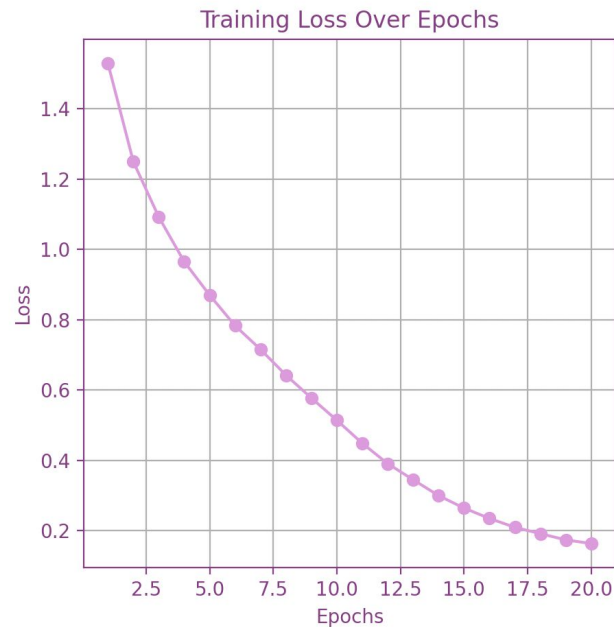
After ~50 epochs

- Accuracy of 92.88%
- Cross Entropy Loss of 0.0772

$$L = -\frac{1}{N} \sum_{i=1}^N \log P(y_i)$$

- Avg 78.34 seconds per Epoch
- Baseline data used for compression and acceleration ratios

Results: Mini-Color NN Data



Progress over first 20 epochs

CNN with Mini-Color:

After ~50 epochs

- Accuracy of 92.01%
- Cross Entropy Loss of 0.0927

$$L = -\frac{1}{N} \sum_{i=1}^N \log P(y_i)$$

- Avg 61.22 seconds per Epoch
- Compression ratio: ~1.7x
 - 186.2 MB / 105.2 MB
- Acceleration ratio: ~1.3x

Conclusion + Future Work

- Results may vary with different neural networks' weights and biases
- Images with many colors may experience significant compromises
 - Could be rendered unreadable
- Future Work
 - Find ways to efficiently apply the quantization. (K-means++)
 - Fine tune the neural network weights to accommodate the new images

Thank you!
Questions?

References

1. K. Ando et al. (2017). *Color image quantization using adaptive spatial color indexing*. IEEE Transactions on Consumer Electronics, 63(3), 264–271. <https://doi.org/10.1109/TCE.2017.8171208>
2. Balasubramanian, V. N., Venkatesh, R., & Sheorey, T. (2019). *Deep color quantization*. In 2019 IEEE Winter Conference on Applications of Computer Vision (WACV) (pp. 1697–1705). IEEE. <https://doi.org/10.1109/WACV.2019.00185>
3. Moroney, N. (2023). *cq100: A high-quality image dataset for color quantization*. Journal of Electronic Imaging, 32(3), 033019. <https://doi.org/10.1117/1.JEI.32.3.033019>
4. SciPython. (n.d.). *Floyd–Steinberg dithering*. SciPython.com. Retrieved May 20, 2025, from <https://scipython.com/blog/floyd-steinberg-dithering/>
5. SuperAnnotate. (n.d.). *Image classification: The basics*. SuperAnnotate Blog. Retrieved May 20, 2025, from <https://www.superannotate.com/blog/image-classification-basics>
6. Vitalflux. (n.d.). *Different types of CNN architectures explained with examples*. Vitalflux.com. Retrieved May 20, 2025, from <https://vitalflux.com/different-types-of-cnn-architectures-explained-examples>